Aim: Implement AVL tree.

CODE:

```
#include <algorithm>
#include <iostream>
using namespace std;
template <typename T> class AVLNode{
	public:
		T key;
		AVLNode *left;
		AVLNode *right;
		int height;
		AVLNode(T k): key(k), left(NULL), right(NULL), height(1) {}
	};
template <typename T> class AVLTree{
	private:
		AVLNode<T> *root;s
		int height(AVLNode<T> *node){
			if(node == NULL)
				return 0;
			return node->height;
		}
		int balanceFactor(AVLNode<T> *node){
			if(node == NULL)
				return 0;
			return height(node->left) - height(node->right);
		}
		AVLNode<T> *rightRotate(AVLNode<T> *y){
			AVLNode<T> *x = y->left;
			AVLNode<T> *T2 = x->right;
			x->right = y;
			y->left = T2;
			y->height = max(height(y->left), height(y->right)) + 1;
			x->height = max(height(x->left), height(x->right)) + 1;
			return x;
		}
		AVLNode<T> *leftRotate(AVLNode<T> *x){
			AVLNode<T> *y = x->right;
			AVLNode<T> *T2 = y->left;
			y->left = x;
			x->right = T2;
			x->height = max(height(x->left), height(x->right)) + 1;
			y->height = max(height(y->left), height(y->right)) + 1;
			return y;
		}

		AVLNode<T> *insert(AVLNode<T>* node, T key){
			if(node == NULL)
				return new AVLNode<T>(key);
```

```
        if(key < node->key)
                node->left = insert(node->left, key);
        else if(key > node->key)
                node->right = insert(node->right, key);
        else
                return node;
        node->height = 1 + max(height(node->left), height(node->right));
        int balance = balanceFactor(node);

        if(balance > 1 && key < node->left->key)
                return rightRotate(node);
        if(balance < -1 && key > node->right->key)
                return leftRotate(node);
        if(balance > 1 && key > node->left->key){
                node->left = leftRotate(node->left);
                return rightRotate(node);
        }
        if(balance < -1 && key < node->right->key){
                node->right = rightRotate(node->right);
                return leftRotate(node);
        }
        return node;
}
AVLNode<T>* minValueNode(AVLNode<T>* node){
        AVLNode<T>* current = node;
        while(current->left != NULL)
                current = current->left;
        return current;
}
AVLNode<T>* deleteNode(AVLNode<T>* root, T key){
        if(root == NULL)
                return root;
        if(key < root->key)
                root->left = deleteNode(root->left, key);
        else if(key > root->key)
                root->right = deleteNode(root->right, key);
        else{
                if((root->left == NULL) || (root->right == NULL)){
                        AVLNode<T>* temp = root->left ? root->left : root->right;
                        if(temp == NULL){
                                temp = root;
                                root = NULL;
                        }
                        else
                                *root = *temp;
                        delete temp;
                }
                else{
```

```
                        AVLNode<T> *temp = minValueNode(root->right);
                        root->key = temp->key;
                        root->right = deleteNode(root->right, temp->key);
                    }
                }
                if(root == NULL)
                    return root;
                root->height = 1 + max(height(root->left), height(root->right));
                int balance = balanceFactor(root);

                if(balance > 1 && balanceFactor(root->left) >= 0)
                    return rightRotate(root);
                if(balance > 1 && balanceFactor(root->left) < 0){
                    root->left = leftRotate(root->left);
                    return rightRotate(root);
                }
                if(balance < -1 && balanceFactor(root->right) <= 0)
                    return leftRotate(root);
                if(balance < -1 && balanceFactor(root->right) > 0){
                    root->right = rightRotate(root->right);
                    return leftRotate(root);
                }
                return root;
            }
            void inorder(AVLNode<T>* root){
                if(root != NULL){
                    inorder(root->left);
                    cout<<root->key<<" ";
                    inorder(root->right);
                }
            }
            bool search(AVLNode<T>* root, T key){
                if(root == NULL)
                    return false;
                if(root->key == key)
                    return true;
                if(key < root->key)
                    return search(root->left, key);
                return search(root->right, key);
            }

    public:
            AVLTree() : root(NULL){}
            void insert(T key){ root = insert(root, key); }
            void remove(T key){ root = deleteNode(root, key); }
            bool search(T key){ return search(root, key); }
            void printInorder(){ inorder(root); cout<<endl; }
    };
```

```
int main(){
	AVLTree<int> tree;
	int n,val;
	cout<<"Enter number of nodes to insert: ";
	cin>>n;
	cout<<"Enter the values: ";
	for(int i=0;i<n;i++){
		cin>>val;
		tree.insert(val);
	}
	cout<<"Inorder Traversal of AVL Tree: ";
	tree.printInorder();

	int del;
	cout<<"Enter value to delete: ";
	cin>>del;
	tree.remove(del);
	cout<<"Inorder Traversal after deletion: ";
	tree.printInorder();

	int s;
	cout<<"Enter value to search: ";
	cin>>s;
	if(tree.search(s))
		cout<<s<<" found in the tree"<<endl;
	else
		cout<<s<<" not found in the tree"<<endl;

	return 0;
}
```

OUTPUT: